Darstellungen ganzer Zahlen

Eine Ausarbeitung zum Vortrag

Proseminar Kombinatorik

Sommersemester 2026

Carl-Phlip Deerberg

carl-philip.deerberg@uni-bielefeld.de

14. April 2026

Inhaltsverzeichnis

1 Einleitung **1**

2 b-adische Darstellung natürlicher Zahlen **2**

3 b-Komplementdarstellung ganzer Zahlen **5**

Literatur **13**

1 Einleitung

Die Ausarbeitung befasst sich mit Darstellungsformen von Zahlen, insbesondere ganzer Zahlen und basiert auf [1].

Während für Menschen die Menge der ganzen Zahlen zwar eine nicht überschaubar große Menge an Zahlen darstellt, so sind die Elemente der Menge jedoch gut darstellbar. Nehmen wir ein beliebiges Element der ganzen Zahlen, so ist dieses in unserer schriftlichen Darstellung eindeutig, für jeden verständlich und leicht reproduzierbar. Gleich ob es eine Zahl in vorstellbarer Größenordnung wie "2" oder "8" ist, oder eine tendenziell schwerer vorstellbare Zahl wie "27439" ist, diese Zahlen sind alle übersichtlich in Schritform festzuhalten.

Von dieser für uns gewohnten Darstellung ausgehend können wir uns nun die Frage stellen, welche Alternativen es geben kann, die Elemente der ganzen Zahlen darzustellen. Insbesondere betrachten wir die Frage, in welcher Form Computer Zahlen darstellen, speichern und verarbeiten.

Um diese Frage zu beantworten müssen wir auf die Ebene der Datenverarbeitung in einem Computer blicken. Für diese digitale Verarbeitung beliebiger Daten hat ein Computer nur die Möglichkeit, diese in Form von Bits zu speichern. Jedes dieser Bits kann nur zwei Zustände annehmen; 0 oder 1. Also schauen wir uns im Folgenden an, wie man ausschließlich mit Nullen und Einsen Zahlen darstellen kann.

Grundsätzliche Definitionen wie die Menge der ganzen oder natürlichen Zahlen werden in dieser Ausarbeitung nicht ausführlich behandelt. Lediglich wird bemerkt, dass es für die folgenden Darstellungen irrelevant ist, ob wir von der Definition $\mathbb{N} = \{1, 2, 3, \dots\}$ ausgehen, oder ob wir die alternative Definition $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ verwenden. Die hier diskutierten Sätze funktionieren in beiden Fällen, auch wenn ich von der erstgenannten Definition ausgehe.

2 b-adische Darstellung natürlicher Zahlen

Bevor wir die Frage für beliebige ganze Zahlen beantworten, betrachten wir zunächst eine übliche Darstellungsform für natürliche Zahlen, die sogenannte b-adische Darstellung. Mathematisch lässt sich diese durch den folgenden Satz beschreiben:

Satz 1. *Sei $b \in \mathbb{N}, b \geq 2$ und $n \in \mathbb{N}$.*

Dann existieren $l \in \mathbb{N}$ und $z_i \in \{0, \dots, b-1\}, i = 0, \dots, l-1$, welche die Bedingungen $z_{l-1} \neq 0$ und

$$n = \sum_{i=0}^{l-1} z_i b^i \tag{1}$$

*erfüllen. l und z_i sind dabei eindeutig bestimmt. Dabei heißt dann $z_{l-1} \dots z_0$ die **b-adische Darstellung** von n und man schreibt auch $n = (z_{l-1} \dots z_0)_b$.*
Dabei gilt:

$$l - 1 = \lfloor \log_b n \rfloor. \tag{2}$$

Für den Beweis des Satzes benötigen wir das folgende Lemma, siehe [1, S. 17, Lemma 1.19]:

Lemma 2. *Sei $k \in \mathbb{N}$ und $l \in \mathbb{N}$. Definiere $f^l \colon \{0, \dots, k-1\}^l \to \{0, \dots, k^l - 1\}$ durch $f^l(w) := \sum_{i=1}^{l} a_i k^{l-i}$ für alle $w = a_1 \dots a_l \in \{0, \dots, k-1\}^l$. Dann ist f^l wohldefiniert und bijektiv.*

Beweis. Siehe [1, S. 17f, Lemma 1.19] □

Beweis. von Satz 1.

Wir beweisen zunächst die Aussage (2).

Falls gilt, dass $n = \sum_{i=0}^{l-1} z_i b^i$ mit $z_i \in \{0, \dots, b-1\}$ für $i = 0, \dots, l-1$ und $z_{l-1} \neq 0$, also n darstellbar als:

$$n = z_{l-1} b^{l-1} + z_{l-2} b^{l-2} + \dots + z_0,$$

so können wir n nach unten abschätzen:

$$n \geq z_{l-1} b^{l-1} \geq 1 \cdot b^{l-1} = b^{l-1}.$$

Da $z_i \leq b-1$, gilt

$$n = \sum_{i=0}^{l-1} z_i b^i \leq \sum_{i=0}^{l-1} (b-1) b^i = (b-1) \underbrace{\sum_{i=0}^{l-1} b^i}_{\text{geometrische Reihe}} = (b-1)\frac{b^l - 1}{b-1} = b^l - 1$$

$$\Rightarrow n \leq b^l - 1$$

$$\Rightarrow n < b^l$$

Es gilt also $b^{l-1} \leq n < b^l$. Wir wenden den Logarithmus auf die Terme an:

$$\log_b(b^{l-1}) \leq \log_b(n) < \log_b(b^l).$$

Da $\log_b(b^x) = x$ gilt, folgt

$$l - 1 \leq \log_b(n) < l,$$

also

$$l - 1 = \lfloor \log_b n \rfloor.$$

Beweis der Eindeutigkeit

Die Eindeutigkeit folgt aus der Bijektivität der Abbildung f^l aus Lemma 2. Die Länge l ist gegeben durch $l = \lfloor \log_b(n) \rfloor + 1$ für festes n. Da f^l injektiv ist, gibt es genau eine Folge $(z_i)_{i=0,\ldots,l-1}$, welche $n = \sum_{i=0}^{l-1} z_i b^i$ erfüllt. Die Bedingung $z_{l-1} \neq 0$ gilt automatisch mit der Wahl von l, da $n \geq b^{l-1}$.

Beweis der Existenz

Wir beweisen die Existenz der b-adischen Darstellung durch vollständige Induktion über $l(n) := 1 + \lfloor \log_b(n) \rfloor$. Eine Erläuterung der vollständigen Induktion findet sich in [1, S. 26].

Induktionsanfang:

$$l(n) = 1 = 1 + \lfloor \log_b(n) \rfloor \Rightarrow \lfloor \log_b(n) \rfloor = 0$$

Das gilt für alle n mit $b^0 \leq n < b^1 \Rightarrow n \in \{1, \ldots, b-1\}$. Dann ist die Darstellung $n = \sum_{i=0}^{0} z_i b^i = z_0 b^0 = z_0$ trivial.

Induktionsschritt: $l - 1 \to l$.

Sei $n \in \mathbb{N}$ mit $l(n) = l \geq 2$ und wir setzen $n' := \lfloor \frac{n}{b} \rfloor$. Für $l' = l(n')$ gilt

$$l' = \left\lfloor \log_b \left\lfloor \frac{n}{b} \right\rfloor \right\rfloor + 1 = \left\lfloor \log_b(\frac{n}{b}) \right\rfloor + 1 = \lfloor \log_b n \rfloor - 1 + 1 = l - 1$$

Dann existiert nach Induktionsvoraussetzung für n' eine Darstellung

$$n' = \sum_{i=0}^{l'-1} z'_i b^i \text{ mit } z'_i \in \{0, \dots, b-1\} \text{ und } z'_{l'-1} \neq 0$$

Es gilt mit $z_0 := n \bmod b \in \{0, \dots, b-1\}$:

$$n = b \cdot \left\lfloor \frac{n}{b} \right\rfloor + (n \bmod b) = b \cdot n' + z_0$$

Einsetzen der Induktionsvoraussetzung ergibt

$$n = b \cdot \left(\sum_{i=0}^{l'-1} z'_i b^i \right) + z_0 = \left(\sum_{i=0}^{l'-1} z'_i b^{i+1} \right) + z_0 \underbrace{=}_{j:=i+1} \left(\sum_{j=1}^{l'} z'_{j-1} b^j \right) + z_0$$

Da $l' = l - 1$, setze $z_j := z'_{j-1}$ für $j \in \{1, \dots, l-1\}$ und wir erhalten

$$n = \sum_{j=1}^{l-1} z_j b^j + z_0 b^0 = \sum_{j=0}^{l-1} z_j b^j$$

□

Beispiel 3. *Die Zahl* 106 *lässt sich, ähnlich zur üblichen Dezimaldarstellung, b-adisch wie folgt schreiben:*

$$\begin{aligned} 106 &= 1 \cdot 10^2 + 0 \cdot 10^1 + 6 \cdot 10^0 \\ &= 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 \end{aligned}$$

bzw. dann in für Computer verarbeitbarer Binärdarstellung 1101010. Wie wir in Satz 1 gesehen haben, kann man jede beliebige Zahl $b \geq 2$ als Basis für die Darstellung der natürlichen Zahl nehmen.

Die Rechenoperationen innerhalb der natürlichen Zahlen funktionieren innerhalb der Binärdarstellung ähnlich der üblichen Operationen wie im gewöhnlichen System.

Folgendes sei noch kurz anzumerken: Die Wahl von b mit der entsprechenden Einschränkung folgt aus der praktischen Relevanz für die Datenverarbeitung: Computer nutzen außer der bereits diskutierten Binärdarstellung ($b = 2$) die Oktaldarstellung ($b = 8$) und auch die Hexadezimaldarstellung ($b = 16$).

3 b-Komplementdarstellung ganzer Zahlen

Nachdem wir uns die Darstellung der natürlichen Zahlen angeschaut haben, stellen wir uns die Frage, wie wir die ganzen Zahlen entsprechend darstellen können. Bei unserer gewohnten Darstellung besteht der Unterschied zwischen den natürlichen Zahlen und den ganzen Zahlen im wesentlichen lediglich in dem vorangestellten Vorzeichen: Für die ganzen Zahlen stellen wir jedem Element der natürlichen Zahlen ein "$-$" voran und erhalten die Menge $\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$

Lässt sich dieses System auch analog auf eine für Computer verarbeitbare Darstellung übertragen? Schaut man sich das oben betrachtete System aus Bits, welche die Zustände 0 und 1 annehmen können, wäre es so etwa möglich, das erste Bit für das jeweilige Vorzeichen zu reservieren. Man spricht hier dann von einer sogenannten Vorzeichen- oder auch Betragsdarstellung von Zahlen.

Beispiel 4. *Eine einfache Darstellung wäre hier eine Binärdarstellung mit vier Bits. Das erste Bit verwenden wir, um das Vorzeichen der mit den weiteren drei Bits folgenden Zahl darzustellen. Die* 0 *im ersten Bit steht dabei für ein positives Vorzeichen, die* 1 *für ein negatives. So ist* $+5 \Leftrightarrow 0101$ *und* $-5 \Leftrightarrow 1101$. *Durch diese Darstellung lassen sich die Zahlen* -7 *bis* $+7$ *wie folgt darstellen:*

$$\begin{aligned} 0 \Leftrightarrow 0000 \ ; \ & -0 \Leftrightarrow 1000 \\ 1 \Leftrightarrow 0001 \ ; \ & -1 \Leftrightarrow 1001 \\ & \vdots \\ 7 \Leftrightarrow 0111 \ ; \ & -7 \Leftrightarrow 1111 \end{aligned}$$

Ein Nachteil ist hier bereits offensichtlich: Es gibt 2 verschiedene Darstellungen für die 0. Während dies jedoch in der Datenverarbeitung vernachlässigbar wäre, stellt sich ein Problem bei der Rechenoperation der Addition heraus. Bei der Binärdarstellung für natürliche Zahlen, vergleiche Beispiel 3, funktioniert die Addition Bitweise wie bei der gewöhnlichen, schriftlichen Addition spaltenweise. Dort ergibt beispielsweise $0001 + 0001 = 0010$, also $1 + 1 = 2$. Dementsprechend muss im Beispiel 4 dann $0010 + 1001 = 0001$, also $2 + (-1) = 1$, ergeben.

Aufgrund dieser Schwierigkeit wird im Regelfall eine andere Darstellung für die ganzen Zahlen verwendet.

Man nimmt jede negative Zahl $z \in \{-2^{l-1}, \dots, -1\}$ mit $z + 2^l$. l steht hier für die Anzahl der verwendeten Bits. Da jedes Bit zwei Zustände annehmen

kann, lassen sich 2^l Zahlen mit l Bits darstellen. Da auch bei dieser Darstellung prinzipiell ein Bit für das Vorzeichen reserviert wird, ist dieses Bit nicht zur Darstellung des Betrags der Zahl verfügbar. Dementsprechend reduziert sich die Anzahl der darstellbaren Zahlen mit l Bits auf 2^{l-1}.

Beispiel 5. *Bei 4 verfügbaren Bits, also $l = 4$, wird beispielsweise die Zahl $z = -3$ wie folgt dargestellt:*

$$z + 2^l = -3 + 16 = 13$$

13 *wird in Binärform zu* 1101. *Da das erste Bit eine* 1 *ist, hat dieses einen negativen Wert und man interpretiert die Bits wie folgt:*

$$\begin{aligned}
&\textit{Bit 3: } -2^3 = -8 \\
&\textit{Bit 2: } 2^2 = 4 \\
&\textit{Bit 1: } 2^1 = 2 \\
&\textit{Bit 0: } 2^0 = 1
\end{aligned}$$

Konkret für 1101 *bedeutet dies*

$$(1 \cdot (-8)) + (1 \cdot 4) + (0 \cdot 2) + (1 \cdot 2) = -3$$

Daraus folgt, dass sich mit vier Bits die folgenden Zahlen darstellen lassen:

Beispiel 6.

$$\begin{aligned}
0 \Leftrightarrow 0000 \ ; \ & -8 \Leftrightarrow 1000 \\
1 \Leftrightarrow 0001 \ ; \ & -7 \Leftrightarrow 1001 \\
&\vdots \\
7 \Leftrightarrow 0111 \ ; \ & -1 \Leftrightarrow 1111
\end{aligned}$$

Die Addition funktioniert hier auch wie bereits beschrieben:

$$\begin{aligned}
&\ \ 0010 \ \textit{also } 2 \\
&+ 1001 \ \textit{also } -7 \\
&= 1011 \ \textit{also } -5
\end{aligned}$$

Wir wollen die b-Komplementdarstellung einer Zahl nun allgemein und für

beliebige Basen $b \geq 2$ definieren.

Zunächst definieren wir hierfür das b-Komplement in allgemeiner Form:

Definition 7. *Seien $l, b \in \mathbb{N}$, mit $b \geq 2$, sei $n \in \{0, \ldots, b^l - 1\}$. Dann ist*

$$K_b^l(n) := (b^l - n) \bmod b^l$$

das l-stellige b-Komplement von n. Für die Basis $b = 2$ nennt man K_2 das Zweierkomplement, für $b = 10$ nennt man K_{10} das Zehnerkomplement.

Weiter beschreiben wir allgemein, wie man das Komplement einer Zahl berechnet.

Lemma 8. *Seien $l, b \in \mathbb{N}$, mit $b \geq 2$. Sei $n = \sum_{i=0}^{l-1} z_i b^i$ mit $z_i \in \{0, \ldots, b-1\}$ für $i = 0, \ldots, l-1$. Dann gilt:*

(i) $K_b^l(n+1) = \sum_{i=0}^{l-1} (b - 1 - z_i) b^i$, *falls* $n \neq b^l - 1$

(ii) $K_b^l(0) = 0$

(iii) $K_b^l(K_b^l(n)) = n$

Beweis. **(ii).** Folgt aus Definition 7: $K_l^b(n) = (b^l - n) \bmod b^l$, mit $n = 0$:

$$K_l^b(0) = (b^l - 0) \bmod b^l = b^l \bmod b^l = 0$$

(i) Sei nun $n \in \{0, \ldots, b^l - 2\}$. Es gilt nach Definition 7: $K_l^b(m) = (b^l - m) \bmod b^l$. Da $0 \leq n \leq b^l - 2$ ist, folgt für $m = n + 1$: $1 \leq n + 1 \leq b^l - 1$. Das heißt dann für $(b^l - (n+1))$ gilt $1 \leq b^l - (n+1) \leq b^l - 1$, liegt also in $\{1, \ldots, b^l - 1\}$. Es folgt, dass die Modulo-Operation trivial ist:

$$K_l^b(n+1) = b^l - (n+1) = b^l - 1 - n.$$

Es gilt wegen der geometrischen Reihe: $b^l - 1 = \sum_{i=1}^{l-1} (b-1) b^i$, also

$$\begin{aligned} K_l^b(n+1) &= \left(\sum_{i=0}^{l-1} (b-1) b^i \right) - n \\ &= \left(\sum_{i=0}^{l-1} (b-1) b^i \right) - \left(\sum_{i=0}^{l-1} z_i b^i \right) \end{aligned}$$

$$= \sum_{i=0}^{l-1}((b-1)b^i - z_i b^i)$$
$$= \sum_{i=0}^{l-1}(b-1-z_i)b^i$$

(iii) Für $n = 0$ gilt nach (ii):

$$K_l^b(K_l^b(0)) = K_l^b(0) = 0$$

Sei nun $n \in \{1, \ldots, b^l - 1\}$ und wir setzen $m := K_l^b(n)$. Wegen der Wahl von n gilt dann

$$m = (b^l - n) \bmod b^l = b^l - n.$$

Wir wenden K_l^b an:

$$K_l^b(m) = (b^l - m) \bmod b^l$$

und substituieren $m = b^l - n$:

$$K_l^b(m) = (b^l - (b^l - n)) \bmod b^l = n \bmod b^l.$$

Da $0 < n < b^l$, gilt $n \bmod b^l = n$ und es folgt

$$K_l^b(K_l^b(n)) = n.$$

□

Wir betrachten das folgende Beispiel und nutzen das Lemma 8 (i), um das b-Komplement von positiven Zahlen zu berechnen. Prinzipiell subtrahiert man erst 1 von der ursprünglichen Zahl und berechnet dasnn stellenweise die Differenz zu $b - 1$:

Beispiel 9. *1. Sei $b = 10, l = 4$. Wir berechnen $K_4^2(4809)$.*

(a) n bestimmen: $n + 1 = 4809 \Rightarrow n = 4809 - 1 = 4808$

(b) n in Ziffern z_i zerlegen: $n = 4 \cdot 10^3 + 8 \cdot 10^2 + 0 \cdot 10^1 + 8 \cdot 10^0$.
Also: $z_3 = 4, z_2 = 8, z_1 = 0, z_0 = 8$.

(c) *Einsetzen ergibt:*

$$\begin{aligned} K_4^1 0(4808+1) &= \sum_i = 0^3(10-1-z_i)10^i \\ &= (9-4)\cdot 10^3 + (9-8)\cdot 10^2 \\ &\quad + (9-0)\cdot 10^1 + (9-8)\cdot 10^0 \\ &= 5\cdot 10^3 + 1\cdot 10^2 + 9\cdot 10^1 + 1\cdot 10^0 \end{aligned}$$

Also ist $K_4^1 0(4809) = (5191)_1 0$.

2. *Sei* $b = 2, l = 4$. *Wir berechnen* $K_4^2(6)$. 6 *entspricht* $(0110)_2$.

   (a) n *bestimmen:* $n + 1 = (0110)_2 \Rightarrow n = (0110)_2 - 1 = (0101)_2$

   (b) n *in Ziffern* z_i *zerlegen:* $n = 0\cdot 2^3 + 1\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0$.
   Also: $z_3 = 0, z_2 = 1, z_1 = 0, z_0 = 1$.

   (c) *Einsetzen ergibt:*

$$\begin{aligned} K_4^2((0110)_2) &= \sum_{i=0}^{3}(2-1-z_i)2^i = \sum_{i=0}^{3}(1-z_i)2^i \\ &= (1-0)\cdot 2^3 + (1-1)\cdot 2^2 + (1-0)\cdot 2^1 + (1-1)\cdot 2^0 \\ &= 1\cdot 2^3 + 0\cdot 2^2 + 1\cdot 2^1 + 0\cdot 2^0 \\ &= (1010)_2 \end{aligned}$$

Also ist $K_4^2((0110)_2) = (1010)_2$.

Nun können wir ganze Zahlen in der b-Komplementdarstellung darstellen und speichern. Damit Computer adäquat damit rechnen können, ist es vorteilhaft, wenn diese Zahlen dabei eine einheitliche Struktur aufweisen. Konkret geht es darum, beliebige positive oder negative ganze Zahlen in einem festgelegten Speicherbereich von l Stellen unterbringen zu können. Möglich gemacht wird diese einheitliche Darstellung durch die folgende Definition:

Definition 10. *Seien* $b, l \in \mathbb{N}$ *mit* $b \geq 2$ *und sei* $n \in \left\{-\left\lfloor \frac{b^l}{2} \right\rfloor, \ldots, \left\lceil \frac{b^l}{2} \right\rceil - 1\right\}$. *Man erhält die* l*-stellige* b*-Komplementdarstellung von* n*, indem man die* b*-adische Darstellung von* n*, falls* $n \geq 0$*, bzw. die* b*-adische Darstellung von* $K_b^l(-n)$*, falls* $n < 0$*, von der vordersten Stelle mit so vielen Nullen auffüllt, bis sich eine* l*-stellige Zahl ergibt.*

Mit dieser Darstellungsform kann der Computer alle Zahlen einheitlich speichern. Darüber Hinaus ist der Vorteil der b-Komplementdarstellung, dass wir bzw. der Computer mit dieser ohne Fallunterscheidungen rechnen können. Es ist also irrelevant, ob wir positive oder negative ganze Zahlen vorliegen haben. Einfach gesprochen können wir in der b-Komplementdarstellung Addition und Subtraktion so durchführen, als ob wir immer positive ganze Zahlen vorliegen hätten.

Der folgende Satz begründet dabei, dass sich alle negativen ganzen Zahlen so in positive Zahlen im Sinne der b-Komplementdarstellung umwandeln lassen, um so wie oben beschrieben rechnen zu können:

Satz 11. *Seien $b, l \in \mathbb{N}$ mit $b \geq 2$. Setze $Z := \left\{ -\left\lfloor \frac{b^l}{2} \right\rfloor, \ldots, \left\lceil \frac{b^l}{2} \right\rceil - 1 \right\}$. Definiere die Funktion $f\colon Z \to \{0, \ldots, b^l - 1\}, z \mapsto \begin{cases} z & \text{falls } z \geq 0 \\ K_l^b(-z) & \text{falls } z < 0 \end{cases}$. Dann ist f bijektiv und es gilt für $x, y \in Z$:*

(i) Ist $x + y \in Z$, so gilt

$$f(x + y) = (f(x) + f(y) \bmod b^l).$$

(ii) Ist $x \cdot y \in Z$, so gilt

$$f(x \cdot y) = (f(x) \cdot f(y)) \bmod b^l.$$

Beweis. **Bijektivittät** Wir betrachten Definitions- und Zielbereich von f. Die Menge

$$Z = \left\{ -\left\lfloor \frac{b^l}{2} \right\rfloor, \ldots, \left\lceil \frac{b^l}{2} \right\rceil - 1 \right\}$$

enthält genau b^l Elemente. Die Zielmenge

$$\{0, \ldots, b^l - 1\}$$

enthält ebenfalls genau b^l Elemente. Wir zeigen die Injektivität von f:

Seien $z_1, z_2 \in Z$. Angenommen, $f(z_1) = f(z_2)$. Da $f(z) \equiv z (\mathrm{mod} b^l)$, gilt $z_1 \equiv z_2 (\mathrm{mod} b^l)$. Da die Differenz von zwei verschiedenen Elementen aus

Z im Betrag kleiner als b^l sein muss, kann die Bedingung nur gelten, wenn $z_1 = z_2$. Es folgt: f ist injektiv.

Da f injektiv ist, und $|Z| = b^l = |\{0, \ldots, b^l - 1\}|$, ist f bijektiv.

Seien $x, y \in Z$ und $p, q \in \{0, 1\}$. Wir betrachten $f(x) = x + p \cdot b^l$ und $f(y) = y + q \cdot b^l$.

(i) Beh.: $f(x + y) = (f(x) + f(y)) \bmod b^l$ gilt, wenn $(x + y) \in Z$.

Es gilt:

$$\begin{aligned}
(f(x) + f(y))(\mathrm{mod} b^l) &= ((x + p \cdot b^l) + (y + q \cdot b^l))(\mathrm{mod} b^l) \\
&= (x + y + p \cdot b^l + q \cdot b^l)(\mathrm{mod} b^l) \\
&= (x + y + \underbrace{(p + q) \cdot b^l}_{\text{Vielfaches von } b^l})(\mathrm{mod} b^l) \\
&= (x + y)(\mathrm{mod} b^l) \\
&= (x + y + b^l)(\mathrm{mod} b^l) \\
&= f(x + y)
\end{aligned}$$

(ii) Beh.: $f(x \cdot y) = (f(x) \cdot f(y)) \bmod b^l$ gilt, wenn $(x \cdot y \in Z)$.

Es gilt:

$$\begin{aligned}
f(x + y) &= (x + y + b^l) \bmod b^l \\
&= (x + y) \bmod b^l \\
&= (x + y + 0) \bmod b^l \\
&= (x + y + \underbrace{(p + 1) \cdot b^l}_{\equiv 0 (\bmod b^l)}) \bmod b^l \\
&= (x + p \cdot b^l + y + q \cdot b^l) \bmod b^l \\
&= ((x + p \cdot b^l) + (y + q \cdot b^l)) \bmod b^l \\
&= (f(x) + f(y)) \bmod b^l
\end{aligned}$$

□

Mit den hier eingeführten Darstellungen für natürliche und ganze Zahlen haben wir Möglichkeiten gesehen, die Zahlen auf alternative Art und Weise darzustellen. Wie bereits an mehreren Stellen erwähnt, bieten diese Formen besonders

für die Verarbeitung in Computern Vorteile. Aufgrund der diskutierten Sätze und Eigenschaften, können wir die gewöhnlichen Rechenoperationen nun auch innerhalb der b-Komplementdarstellung von ganzen Zahlen anwenden, ohne auf Ungenauigkeiten oder Probleme bei der Verarbeitung zu stoßen.

Literatur

[1] Stefan Hougardy und Jens Vygen. Algorithmische Mathematik. 2., korrigierte und erweiterte Auflage. Berlin: Springer Spektrum, 2018.